

EJERCICIO GUIADO + TRABAJO EN CASA

Aplicación Web Monolítica para Gestión de una Librería

GCP SDK CLI | Compute Engine | CentoOS 10 Stream | Node.js | Express | EJS | PostgreSQL | 4FN | CRUD | Uploads | Autenticación | Apache / NGINX | ubiquitous.udem.edu

Nombre	
Matrícula	
Grupo y hora	
Semestre	
Fecha	

EJERCICIO GUIADO

Objetivo

Diseñar, construir, probar, desplegar y documentar una aplicación web monolítica en Node.js para la gestión de una librería en línea, utilizando PostgreSQL mediante acceso directo desde la aplicación. El propósito del ejercicio no es únicamente “hacer que el sistema funcione”, sino tomar y justificar decisiones como ingenieros de tecnologías computacionales: analizar requisitos, modelar datos, seleccionar una arquitectura, organizar el código, proteger la información, validar el sistema, documentar riesgos y publicar evidencias reproducibles del trabajo realizado.

La solución deberá renderizar HTML del lado del servidor, administrar usuarios registrados, implementar CRUD sobre todas las tablas del modelo normalizado, manejar imágenes de libros y permitir registrar conceptos y definiciones asociadas al contenido de cada libro.

Restricciones arquitectónicas del ejercicio

- La solución será monolítica y server-side utilizando Node.js, Express y EJS.
- La aplicación accederá directamente a PostgreSQL mediante el controlador pg y consultas SQL parametrizadas.
- No se desarrollarán APIs REST, GraphQL, SOAP ni microservicios.
- No se utilizarán JSON o XML como mecanismos de intercambio de información entre frontend y backend. package.json se conserva únicamente porque npm lo requiere.
- Las vistas se generarán en el servidor y los formularios HTML enviarán sus datos directamente al monolito.
- Sólo podrán ingresar usuarios registrados y deberá existir como máximo un Administrador.
- El estudiante deberá poder explicar y defender cada decisión arquitectónica, de datos, seguridad, despliegue y organización del código.

Criterio de trabajo como ingeniero

Para cada decisión relevante evita limitarte a “usé esta tecnología porque lo pedía el ejercicio”. Registra el razonamiento mediante el siguiente esquema:

Necesidad o problema → alternativas consideradas → decisión tomada → justificación técnica
→ riesgo o limitación → evidencia de validación

Crea durante el ejercicio un registro de decisiones en docs/ENGINEERING_DECISIONS.md. Este archivo formará parte de la evidencia final publicada en tu página personal.

Parte 1. Analizar el problema antes de programar

1. Definir requisitos funcionales y no funcionales

Antes de diseñar la base de datos o escribir código, analiza el problema de la librería y documenta qué debe hacer el sistema y bajo qué condiciones debe operar.

Como mínimo, identifica los siguientes requisitos funcionales:

- Registro, inicio y cierre de sesión de usuarios.
- Consulta del catálogo de libros por usuarios autenticados.

- Búsqueda por ISBN y título.
- CRUD de libros, autores, géneros, formatos, categorías y conceptos.
- Asociación de múltiples autores y géneros a un mismo libro.
- Registro de conceptos y definiciones específicas por libro.
- Carga, edición y eliminación de imágenes; una imagen podrá marcarse como portada.
- Control de stock y precio.
- Administración restringida al único usuario Administrador.

Identifica también requisitos no funcionales que condicionan el diseño: seguridad, mantenibilidad, integridad de datos, rendimiento básico, usabilidad, disponibilidad, trazabilidad de errores y facilidad de despliegue.

Entrega de esta parte: docs/REQUIREMENTS.md con requisitos numerados (RF-01, RF-02... y RNF-01, RNF-02...), supuestos, restricciones y criterios de aceptación.

2. Identificar actores, operaciones y riesgos

Define al menos los actores Visitante, Usuario Registrado y Administrador. Para cada actor documenta qué operaciones puede ejecutar y cuáles deben rechazarse. Identifica además riesgos iniciales: acceso no autorizado, SQL Injection, subida de archivos peligrosos, exposición de credenciales, eliminación accidental de información y publicación de datos sensibles.

Parte 2. Tomar decisiones de arquitectura y organización del software

3. Diseñar la macro-arquitectura monolítica

Genera un diagrama que muestre el flujo Navegador → Apache/NGINX → Aplicación Node.js/Express → módulos internos → PostgreSQL. Señala claramente que la interfaz, lógica de negocio y acceso a datos pertenecen a una sola unidad desplegable.

Tu diagrama debe identificar al menos: presentación, rutas/controladores, servicios o lógica de negocio, middleware, acceso a datos, vistas, archivos estáticos, PostgreSQL y almacenamiento de imágenes.

Guarda el diagrama en docs/ARCHITECTURE_MONOLITHIC.png y explica por qué esta solución sigue siendo monolítica aunque el código esté organizado en módulos.

4. Seleccionar patrón de presentación y organización del código

Define y justifica cómo organizarás la interfaz y el código. Puedes utilizar un enfoque tipo MVC o una variante server-side equivalente, siempre que exista separación de responsabilidades. Explica qué responsabilidad tendrá cada carpeta y evita concentrar toda la lógica en app.js o en las vistas EJS.

Elemento	Responsabilidad esperada
app.js	Inicialización de Express, middleware general, montaje de rutas y arranque controlado.
config/	Configuración y conexión a PostgreSQL.
routes/	Recepción de solicitudes HTTP y coordinación del flujo.

services/ o controllers/	Reglas de aplicación y operaciones de negocio.
middleware/	Autenticación, autorización, validaciones transversales y manejo de errores.
views/	Plantillas EJS; no deberán contener consultas SQL ni lógica de negocio compleja.
public/	CSS, JavaScript de interfaz, imágenes y otros recursos estáticos.
uploads/	Archivos cargados, con controles de tipo, tamaño y nombre.

5. Registrar las decisiones arquitectónicas

Documenta como mínimo tres decisiones: elección de monolito, acceso directo a PostgreSQL y renderizado server-side con EJS. Para cada una registra beneficios, limitaciones y qué condición futura podría justificar cambiarla.

Parte 3. Diseñar y normalizar la base de datos hasta 4FN

6. Identificar entidades, atributos y dependencias

Partiendo de ISBN, título, autor, año de publicación, género, precio, stock, formato, imágenes y conceptos definidos por libro, identifica entidades, claves candidatas, dependencias funcionales y dependencias multivaluadas.

- Un libro puede tener varios autores y un autor puede participar en varios libros.
- Un libro puede pertenecer a varios géneros y un género puede clasificar varios libros.
- Un libro puede definir muchos conceptos y un mismo concepto puede aparecer en distintos libros con definiciones diferentes.
- Un libro puede tener varias imágenes.
- Formato y categoría se modelarán como catálogos independientes.
- La regla de “un solo Administrador” debe protegerse también en la base de datos.

7. Normalizar de forma demostrable hasta 4FN

No presentes únicamente el modelo final. Documenta la evolución desde una estructura inicial no normalizada hasta 1FN, 2FN, 3FN/BCNF y 4FN. Identifica explícitamente por qué las dependencias multivaluadas de autores, géneros, conceptos e imágenes no deben almacenarse como listas o columnas repetitivas dentro de books.

Genera los siguientes productos:

- docs/NORMALIZATION_4FN.xlsx: template del proceso de normalización utilizado en la clase de Bases de Datos Avanzadas.
- docs/DB_DESIGN_ER_4FN.png: diagrama ER final.
- db/00_create_database.sql: creación de la base de datos/usuario cuando aplique.
- db/01_schema.sql: tablas, PK, FK, UNIQUE, CHECK, índices y restricciones.
- db/02_seed_30_per_table.sql: datos sintéticos suficientes para probar la solución.

8. Diseñar integridad y reglas de negocio en PostgreSQL

Para cada tabla justifica PK, FK, restricciones UNIQUE y CHECK. Define las acciones ON UPDATE/ON DELETE sólo cuando tengan sentido y explica por qué. Implementa una defensa en base de datos que impida tener más de un Administrador y documenta cómo funciona.

Ejecuta pruebas negativas de integridad: ISBN duplicado, stock negativo, precio inválido, FK inexistente, eliminación que viole una relación y creación de un segundo administrador. Conserva la sentencia ejecutada, el resultado esperado y el error real generado por PostgreSQL.

Parte 4. Preparar infraestructura y PostgreSQL en GCP

9. Crear la instancia mediante GCP SDK CLI

Utiliza GCP SDK CLI para crear una instancia de Compute Engine con CentOS Stream 10. Documenta el comando utilizado, región/zona, tamaño de instancia, reglas de firewall necesarias y una breve justificación del dimensionamiento seleccionado para un entorno del ejercicio.

Registra los comandos de infraestructura en docs/GCP_COMMANDS.md. No publiques llaves privadas, tokens ni credenciales.

10. Instalar y configurar PostgreSQL

Instala PostgreSQL, crea la base de datos y un usuario de aplicación con los privilegios mínimos necesarios. No ejecute la aplicación web utilizando un superusuario de PostgreSQL.

Carga los scripts en el orden siguiente y registra evidencia de cada ejecución:

```
db/00_create_database.sql
db/01_schema.sql
db/02_seed_30_per_table.sql
db/03_all_queries_before_stored_procedures.sql
db/04_stored_procedures.sql
db/05_triggers.sql
db/06_views.sql
```

Verifica y documenta sp, triggers, vistas y conteos, relaciones e integridad utilizando psql. Captura únicamente la información técnica necesaria; oculta contraseñas y datos sensibles. No olvides obtener screenshots para tu reporte.

Parte 5. Construir la aplicación monolítica en Node.js

11. Crear la estructura del proyecto

Crea el proyecto Node.js e instala únicamente las dependencias justificadas. Antes de programar, documenta la función de cada directorio y archivo principal. Mantén separación entre configuración, rutas, lógica de negocio, middleware, vistas y recursos estáticos.

La estructura mínima esperada puede aproximarse a:

```
library/
app.js
```

```
package.json
.env                # NO publicar
config/db.js
routes/
services/
middleware/
views/
public/
uploads/
db/
docs/
```

12. Implementar acceso a PostgreSQL de forma segura

Todas las operaciones con datos deberán utilizar consultas parametrizadas con pg. Está prohibido construir SQL concatenando directamente valores proporcionados por el usuario. Centraliza la conexión a PostgreSQL y maneja errores sin mostrar detalles internos de la base de datos al usuario final.

13. Implementar autenticación y autorización

Implementa registro, login, logout y control de sesión. Las contraseñas deberán almacenarse mediante hash seguro; nunca en texto plano. Protege rutas privadas con middleware y separa autorización de autenticación.

- Visitante: sólo puede acceder a login/registro y páginas públicas expresamente autorizadas.
- Usuario registrado: consulta catálogo, detalle de libro y conceptos permitidos.
- Administrador: acceso a CRUD y administración del sistema.
- Un usuario regular que intente acceder a funciones administrativas deberá recibir una respuesta controlada de acceso denegado.

14. Implementar CRUD completo

El Administrador deberá poder crear, consultar, modificar y eliminar los registros de todas las tablas administrables. No basta con que exista una sentencia SQL: debe existir la interfaz web y validación correspondiente, sp, triggers y vistas.

Para cada CRUD verifica: formulario, validación server-side, consulta parametrizada, mensaje de resultado, manejo de error y evidencia visual.

15. Implementar conceptos asociados a libros

El sistema tendrá un apartado específico para administrar definiciones del contenido de cada libro. Por ejemplo, para un libro de Computación sobre Cloud Computing registra conceptos como IaaS, PaaS, SaaS, FaaS, Bucket, Public Cloud, Private Cloud, Hybrid Cloud, Multicloud y Serverless. La definición pertenece a la relación libro-concepto y puede incluir referencia a capítulo o página.

16. Implementar carga de imágenes

Permite cargar imágenes JPG, PNG y WebP mediante multipart/form-data. Valida extensión/tipo permitido, tamaño máximo y nombre de archivo generado por el sistema. Evita usar directamente el nombre enviado por el usuario. Permite marcar una imagen como portada y administrar texto alternativo.

La base de datos debe guardar metadatos y la referencia/ruta del archivo; no expone rutas internas sensibles.

Parte 6. Seguridad de la aplicación

17. Aplicar controles mínimos de seguridad

Revisa y documenta, como mínimo, los siguientes controles:

- Hash de contraseñas y política básica de contraseñas.
- Variables de entorno para secretos y credenciales; .env no deberá subirse ni publicarse.
- Consultas SQL parametrizadas para reducir riesgo de SQL Injection.
- Validación server-side de todos los campos aunque exista validación HTML/JavaScript.
- Autorización por rol en cada ruta administrativa.
- Manejo seguro de sesiones y cierre de sesión.
- Validación de archivos subidos: extensión, MIME, tamaño y nombre.
- Mensajes de error controlados; no mostrar stack traces ni SQL al usuario final.
- Principio de mínimo privilegio para el usuario PostgreSQL.
- No publicar contraseñas, archivos .env, claves SSH, tokens o cadenas de conexión en ubiquitous.udem.edu.

Crea docs/SECURITY_REVIEW.md y para cada control indica: amenaza, control aplicado y evidencia de prueba.

Parte 7. Pruebas como evidencia de ingeniería

18. Diseñar un plan de pruebas

Antes de considerar terminado el sistema, prepara una matriz de pruebas. Cada prueba deberá contener ID, requisito relacionado, precondition, entrada, pasos, resultado esperado, resultado observado, estado y evidencia.

Incluye al menos:

- Pruebas funcionales de login, logout, búsqueda y cada CRUD.
- Pruebas de autorización: visitante, usuario registrado y administrador.
- Pruebas negativas de base de datos y restricciones.
- Pruebas de validación de campos y archivos.
- Pruebas de relaciones libro-autor, libro-género y libro-concepto.
- Prueba de creación de segundo Administrador.
- Prueba de consulta SQL con caracteres especiales para verificar parametrización.
- Prueba del despliegue mediante reverse proxy.
- Pruebas básicas de navegación y usabilidad.

Entrega docs/TEST_PLAN.xlsx o docs/TEST_PLAN.md y conserva screenshots relevantes. Una captura sin explicación no constituye por sí sola una prueba.

Parte 8. Desplegar mediante Apache o NGINX

19. Ejecutar Node únicamente en localhost

La aplicación Node.js deberá escuchar en 127.0.0.1:3000 y no exponerse directamente a Internet. Primero verifica su funcionamiento local en la instancia.

```
http://127.0.0.1:3000/library → prueba local  
http://IP_DEL_SERVIDOR/library → acceso mediante reverse proxy
```

20. Configurar reverse proxy

Configura Apache Web Server o NGINX para publicar /library y reenviar las solicitudes hacia Node.js. Documenta el archivo de configuración utilizado y explica la función del reverse proxy dentro de la arquitectura.

Verifica rutas estáticas, formularios, sesiones, carga de imágenes y redirecciones cuando la aplicación opera bajo el prefijo /library. Registra evidencia del acceso final desde un navegador externo.

Parte 9. Documentar las decisiones de ingeniería

21. Preparar el reporte técnico

El reporte no deberá describir únicamente lo que hiciste. Debe justificar las decisiones tomadas y demostrar que comprendes su impacto.

Incluye como mínimo:

- Descripción del problema y alcance.
- Requisitos funcionales y no funcionales.
- Macro-arquitectura y patrón de presentación seleccionado.
- Organización del código y responsabilidades de los módulos.
- Modelo de datos y normalización hasta 4FN.
- Decisiones sobre integridad y restricciones de PostgreSQL.
- Funcionalidades implementadas y flujo navegador → aplicación → base de datos.
- Seguridad: autenticación, autorización, sesiones, validación, secretos, SQL parametrizado y uploads.
- Estrategia de pruebas y resultados.
- Despliegue mediante Apache/NGINX.
- Limitaciones actuales, riesgos técnicos y posibles mejoras.
- Análisis de qué tendría que cambiar si el sistema evolucionara hacia componentes desacoplados.

Parte 10. Publicar TODO el trabajo en ubiquitous.udem.edu

22. Crear la página web de evidencias

Al finalizar, cada estudiante deberá publicar una página web navegable que documente todo el ejercicio en el espacio asignado en el servidor de la materia. La publicación deberá de contener un enlace a la entrega de todo este ejercicio02 en forma de un archivo .tar.gz. Un documento Word o PDF no sustituye la página web de su ejercicio en su página personal de ubiquitous.udem.edu por lo que NO será tomado en consideración.

La URL de entrega deberá seguir el formato:

```
https://ubiquitous.udem.edu/~iac-matricula/ejercicio02/
```

En el servidor, organiza el contenido dentro de tu directorio público HTML, por ejemplo:

```
~/html/ejercicio02/  
  index.html  
  css/  
  img/  
  evidencias/  
  docs/  
  sql/  
  descargas/ejercicio02.tar.gz
```

23. Estructurar el sitio como reporte de ingeniería

Tu archivo index.html deberá presentar el trabajo de forma ordenada, legible y profesional. Como mínimo incluye las siguientes secciones:

1. Portada: nombre, matrícula, grupo, fecha y nombre del ejercicio.
2. Objetivo y alcance.
3. Análisis de requisitos.
4. Decisiones de arquitectura y diagrama del monolito.
5. Diseño de la base de datos, diagrama ER y explicación de 4FN.
6. Infraestructura GCP y PostgreSQL.
7. Estructura de módulos de Node.js.
8. Funcionalidades implementadas.
9. Seguridad y roles.
10. Plan y resultados de pruebas.
11. Despliegue con Apache/NGINX.
12. Galería de evidencias/screenshots con explicación.
13. Decisiones de ingeniería, problemas encontrados y soluciones aplicadas.
14. Uso documentado de Inteligencia Artificial.
15. Conclusiones y mejoras futuras.
16. Descargas de archivos permitidos del ejercicio.

24. Publicar evidencias y archivos descargables

Incluye enlaces a los artefactos que permitan revisar tu trabajo: diagramas, reporte, scripts SQL, normalización, plan de pruebas y documentación técnica. Incluye el proyecto completo compactado en .tar.gz dentro de descargas/.

IMPORTANTE: antes de comprimir o publicar el proyecto elimina o excluye .env, node_modules, llaves privadas, tokens, contraseñas, archivos de configuración con secretos y cualquier dato sensible. Publica un archivo .env.example con nombres de variables pero sin valores reales.

25. Verificar la publicación como usuario externo

Abre tu URL en una ventana privada/incógnito y verifica que todos los enlaces, imágenes, CSS, documentos y descargas funcionen. No des por terminada la entrega mientras existan enlaces rotos o recursos accesibles sólo desde tu sesión.

Al final de la página muestra claramente: URL del ejercicio, fecha de última actualización y versión del trabajo.

TRABAJO EN CASA

Tarea 2a. Reporte de normalización 4FN

Completa el proceso 1FN → 2FN → 3FN/BCNF → 4FN utilizando el formato Excel visto en Bases de Datos Avanzadas. Identifica dependencias multivaluadas y justifica cada tabla puente. Publica el documento y una síntesis visual en la sección “Diseño de Datos” de tu página.

Tarea 2b. Matriz de pruebas e integridad

Diseña al menos 15 casos de prueba, combinando pruebas positivas y negativas. Relaciona cada prueba con un requisito y publica la matriz de resultados. Selecciona las evidencias más representativas; no conviertas la página en una colección de capturas sin explicación.

Tarea 2c. Evaluación arquitectónica

Redacta aproximadamente 600 palabras explicando por qué el monolito server-side es adecuado o no para este escenario. Compáralo con una solución desacoplada por componentes y con microservicios considerando complejidad operativa, despliegue, escalabilidad, mantenimiento, seguridad y tamaño del equipo. No cambies la arquitectura del ejercicio; analiza sus trade-offs como ingeniero.

Tarea 2d. Revisión de seguridad

Realiza una revisión final de seguridad. Selecciona al menos ocho amenazas o errores posibles y documenta cómo fueron mitigados o qué riesgo residual permanece. Publica una tabla resumida en la página web.

Tarea 2e. Evolución controlada con IA

Utiliza docs/PROMPT_MAESTRO_IA.md para solicitar una mejora pequeña y verificable. Conserva el prompt exacto, la respuesta relevante, los archivos modificados, el riesgo introducido, las pruebas ejecutadas y el resultado. Actualiza AI_PROMPT_HISTORY.md y AI_CHANGELOG.md. La evidencia deberá demostrar que la IA fue utilizada como herramienta de ingeniería y no como sustituto del análisis.

Evidencias obligatorias de la entrega final

- URL pública: <https://ubiquitous.udem.edu/~iac-matricula/ejercicio02/>.

- Diagrama de macro-arquitectura.
- Diagrama ER final y reporte de normalización 4FN.
- Scripts SQL de creación, esquema y datos sintéticos.
- Capturas de psql mostrando restricciones y pruebas de integridad.
- Aplicación funcionando y acceso bajo /library mediante Apache o NGINX.
- CRUD de tablas y libros.
- Relaciones libro-autor, libro-género y libro-concepto.
- Conceptos y definiciones del libro de Cloud Computing.
- Gestión de imágenes.
- Pruebas de autenticación, autorización y administrador único.
- Matriz de pruebas.
- Revisión de seguridad.
- ENGINEERING_DECISIONS.md.
- AI_PROMPT_HISTORY.md y AI_CHANGELOG.md.
- Proyecto compactado en .tar.gz sin secretos ni node_modules.
- Página web final con explicación, no sólo enlaces de descarga.

Criterios de evaluación del trabajo de ingeniería

Criterio	Qué se espera demostrar
Análisis	Comprensión del problema, requisitos, restricciones y riesgos antes de programar.
Diseño	Decisiones justificadas de arquitectura, organización del código y modelo de datos.
Implementación	Funcionalidades completas, código organizado y acceso seguro a datos.
Seguridad	Controles efectivos, pruebas negativas y protección de secretos.
Validación	Pruebas reproducibles y evidencia de resultados.
Despliegue	Aplicación disponible correctamente mediante reverse proxy.
Documentación	Capacidad para explicar por qué se diseñó la solución de esa manera.
Publicación web	Evidencias completas, navegables, ordenadas y accesibles en ubiquitous.udem.edu.
Uso de IA	Trazabilidad, revisión crítica, pruebas y responsabilidad sobre los cambios.

Consideración sobre el uso de Inteligencia Artificial

Se permite y fomenta el uso responsable de herramientas de IA para analizar, generar alternativas, documentar, refactorizar y apoyar las pruebas. Sin embargo, cada estudiante continúa siendo responsable de comprender, revisar, probar, validar, corregir y justificar el resultado. La IA no sustituye la explicación del modelo relacional, la defensa de la arquitectura, las decisiones de seguridad ni la interpretación de las pruebas.

Todo uso relevante de IA deberá quedar documentado. Un estudiante debe poder explicar sin ayuda de la herramienta qué cambió, por qué cambió, qué riesgo podía introducir y cómo verificó que la solución continuaba siendo correcta.